When Does Crossover Help Architecture Search? A Building-Block Rule, a Trap-Function Control, and a Real-Space Negative

Vasili Gavrilov*

2026

Abstract

Evolutionary neural architecture search recombines architectures with crossover, yet random search is famously hard to beat and it is rarely clear whether crossover contributes anything of its own. The idea we test: a NAS-Bench-101 cell is multi-dimensional (a wiring dimension × a labeling dimension, plus a path view), so a crossover that recombines along that structure (H_1) ought to beat a structure-blind one; the null (H_0) is that the real cell space carries no such recombinable building blocks. We give one rule with a positive control and a real-space test, on two dependency-free C testbeds, each with paired significance tests. The rule: *crossover helps exactly when the space carries separable, recombinable building blocks.* Positive control — on concatenated deceptive trap functions, separable by construction, one-point crossover's advantage over mutation grows with the number of blocks, from 1.8× at four blocks to a regime at sixteen where mutation reaches the optimum 28/200 times and crossover 199/200. Real space — on NAS-Bench-101 (423,624 tabulated cells, fitness a lookup, so thousands of paired seeds are cheap), no method *meaningfully* beats random: random and all four crossovers finish within a few tenths of a percent of one another, comparable to the benchmark's own seed-to-seed training noise. Thousands of paired seeds do not buy a large gain; they resolve a consistent *ordering* inside that noise band, and the ordering runs the wrong way for cleverness. Every operator built to recombine the cell's structure — factoring its wiring and labeling dimensions, aligning nodes by role, transplanting whole paths — is consistently *worse* than a structure-blind per-cell uniform crossover, and two are consistently (if slightly) below random at scale. So the real cell space has no building-block structure to exploit along those dimensions; uniform crossover's slim, consistent edge over random is diversity, which the structural priors collapse or dissipate (shown by direct diversity measurement and a population-size sweep). The lesson restates a known one — random search is a strong NAS baseline, and structural cleverness does not beat it — but with a controlled mechanism and a positive control that isolates when recombination genuinely works. Every number regenerates from one command.

1 Introduction

Neural architecture search (NAS) automates a network's shape; methods differ mainly in search strategy — RL [5], gradient relaxation [6], Bayesian optimization, random search, evolution [8]. Evolutionary NAS mutates and recombines architectures, training each candidate by backpropagation [7]. Two facts motivate us: plain random search is a strong baseline [4, 9], and the leading evolutionary method drops crossover entirely, keeping only mutation with aging [7].

Crossover [2] is supposed to recombine *building blocks* — partial solutions that can be assembled across parents. So the operative question is not "does crossover help?" but "does the space

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including a genetic-algorithm optimizer built for industrial use in 2002. Reference implementation: `https://github.com/Anode1/SMBPANN`.

Table 1: Concatenated deceptive traps ($K = 4$ bits/block), mean generations to the global optimum and how many of 200 seeds reach it. Crossover's advantage over mutation grows with the number of blocks M — the signature of a genuine building-block effect — until mutation essentially fails and crossover still solves it.

blocks M	mutation-only	crossover + mutation	speedup
4	9.2 gens, 200/200	5.1 gens, 200/200	1.8×
8	70.8 gens, 200/200	14.2 gens, 200/200	5×
12	257 gens, 185/200	22.6 gens, 200/200	11×
16	363 gens, 28/200	42.3 gens, 199/200	mutation fails

have building blocks it can recombine?" We answer with a rule and two testbeds. A positive control fixes the meaning of the rule: deceptive trap functions, which are separable into blocks by construction, where crossover must and does win. Then the real test: NAS-Bench-101, whose cell is naturally multi-dimensional — a wiring dimension and a labeling dimension, plus a path view — and on which we ask whether crossover operators that respect that structure beat a structure-blind one. Stated as a hypothesis: H_1, a crossover that recombines the cell's building blocks (its wiring and labeling dimensions, its node roles, its paths) beats a structure-blind uniform crossover; the null H_0 is that it does not, because the real cell space carries no such recombinable structure. The intuition behind H_1 is strong, and our trap-function control shows the effect it predicts is real when blocks exist. Yet on NAS-Bench-101 the data does not merely fail to reject H_0: the structure-aware operators are significantly *worse* than the structure-blind one, and worse than random search at scale. (The recombination logic is from a self-modifying back-propagation engine [1]; on NAS-Bench-101 we move fitness onto the table so training noise cannot confound the comparison.)

2 Positive control: crossover on separable building blocks

Before the real space, we fix what a building-block advantage looks like on a problem *designed* to have one. The genome is M concatenated deceptive trap blocks of $K = 4$ bits: a block scores K if all its bits are 1, else $K - 1$ minus its number of ones — a gradient that lures a bit-flipping search toward all-zero, so the all-ones optimum is reachable only by *assembling* correctly-solved blocks. This is exactly what one-point crossover recombines and a mutation-only search cannot climb to [3]. We run elitism + tournament selection with one-point crossover ($p = 0.9$) versus the identical search with mutation only (population 1000, 200 seeds, cap 500 generations), and scan M.

Table 1 is the control: when the space is separable into building blocks, crossover is not marginally but *decisively* better, and the gap widens with the amount of structure until mutation alone cannot solve the problem at all. The operator works. The question for a real search space is whether it supplies such blocks.

3 The real space and its fitness oracle

A NAS-Bench-101 cell is a directed acyclic graph on up to seven nodes: node 0 input, node $n-1$ output, interior nodes labeled `conv1x1`/`conv3x3`/`maxpool3x3`. It has two orthogonal dimensions — *wiring* (the adjacency) and *labeling* (the ops) — and a derived path view. The benchmark [12] tabulates the trained CIFAR-10 accuracy of all 423,624 cells, so fitness is a lookup and no network is trained during search.

The benchmark de-duplicates *isomorphic* graphs, so a recombined child is usually a relabeling of a catalogued cell rather than a verbatim match, and lookup needs a canonical form.

We compute it by brute force: prune nodes lying on no input–output path, then take the lexicographically minimum adjacency-plus-labeling encoding over all $\leq 5! = 120$ interior-node permutations — exact and dependency-free. A self-test confirms it: over 5,000 cells both the verbatim graph and a random relabeling map to the same accuracy (5000/5000 each), and the 423,624 rows collapse to exactly 423,624 canonical keys (no collisions). This same canonicalization supplies the alignment operator's node correspondence. The table is extracted from the official `nasbench_only108.tfrecord` by a small C decoder; best test 94.32% / val 95.06% reproduce the published optima.

4 Operators

Every individual is a seven-node upper-triangular cell. All arms share one population loop, one light mutation (flip an edge or re-roll an op), elitism, and query budget; *only the crossover differs*. Selection is on validation accuracy, reporting test. Four crossovers against a random-search control:

- **flat** — per-cell uniform: each adjacency bit and each op from a uniformly chosen parent. Structure-blind, maximum-diversity (and the "competing-conventions" strawman, since node k need not play the same role in both parents).
- **axis** — dimension-factored: the whole wiring from one parent, the whole labeling from the other.
- **aligned** — role-aligned: match the second parent's interior nodes to the first by role (op and in/out degree), then recombine *node modules* (each node's incoming edges and op together from one aligned parent) — removing the competing-conventions problem [10, 11].
- **path** — subgraph transplant: recombine whole input–output paths, each carrying its parent's ops, greedily under the nine-edge cap (the graph dimension).

Matched compute, paired. Each arm gets a fixed query budget; the random control spends the same on independent random cells; a child that prunes invalid or over-budget is a spent query scored zero. In each trial the control and all arms share the *same* seed (identical start), so a trial is a paired observation; we report paired win/loss and the Wilcoxon signed-rank z of (arm − baseline).

5 Results on the real space

Nothing meaningfully beats random. In absolute terms this is a space where random search is essentially unbeatable: across every budget, random and all four crossovers finish within a few tenths of a percent of one another (see Table 2), a gap comparable to the benchmark's own training noise — the same architecture, trained three times, varies by a similar amount. No crossover produces a gain a practitioner would notice. What the paired, many-seed design buys is not size but resolution: it lets us read a *consistent ordering* inside that noise band, and that ordering — not any large effect — is the finding.

Structure does not help; at scale it slips below random. Table 2 scales to population 50, budget 20,000, 4000 paired seeds; all differences below are small (within the noise floor above) but consistent across the 4000 seeds. **Flat** is the best at *every* budget and beats each structured operator head-to-head; its slim edge over random shrinks as random saturates (z : $48 \to 46 \to 29 \to 13$) but stays consistent (2268/1322 paired wins even at 20,000). The structured operators worsen with compute: **axis** crosses from $z{=}{+}30$ to -28 (losing to random 1230/2559); **path** plateaus almost immediately ($93.904 \to 93.913$ over a $40\times$ budget rise) and

Table 2: NAS-Bench-101, population 50, elite 12, 4000 paired seeds. Mean best test-accuracy (%), and Wilcoxon signed-rank z against random search (positive = above random). All gaps are small — within the benchmark's training noise — but consistent across 4000 seeds. Flat is best at every budget; *axis and path slip just below random at scale*; aligned's edge decays to zero.

budget	random	flat (z)	axis (z)	aligned (z)	path (z)
500	93.645	**93.998** (+48.0)	93.806 (+30.3)	93.917 (+41.8)	93.904 (+44.4)
2000	93.746	**94.072** (+45.8)	93.872 (+22.5)	94.009 (+39.8)	93.911 (+29.9)
8000	93.941	**94.085** (+29.2)	93.883 (−**10.2**)	94.036 (+19.7)	93.912 (−**6.1**)
20000	94.042	**94.088** (+12.5)	93.886 (−**27.6**)	94.044 (+1.9)	93.913 (−**26.0**)

Table 3: Population sweep at fixed budget 2000, 600 paired seeds (elite = pop/4; random control identical, mean 93.743). Mean best test-accuracy (%) and Wilcoxon z vs random: at pop 8 every arm loses to random; the gain rises monotonically with population.

pop	flat (z)	axis (z)	aligned (z)	path (z)
8	93.673 (−4.5)	93.652 (−5.7)	93.673 (−4.2)	93.526 (−12.6)
16	93.791 (+3.5)	93.715 (−1.8)	93.756 (+1.1)	93.706 (−2.7)
32	93.981 (+14.6)	93.823 (+5.9)	93.898 (+10.5)	93.840 (+7.3)
64	**94.122** (+19.3)	93.917 (+11.6)	94.064 (+17.2)	93.957 (+14.2)

ends at z=−26; **aligned** decays from z=42 to 1.9, to break-even. So the structure-blind operator wins throughout, the two most structured (axis, path) fall below random at scale, and the intermediate one ties it. This is not a validity artifact: axis/aligned/path yield *more* valid offspring (95–97%) than flat (90–96%).

Mechanism: productive diversity. Two independent measurements locate the cause. (i) Final-population genotypic spread (mean pairwise distance over 21 edges + 5 ops) shows two failure modes: the operators that end below random, axis and path, *collapse* the spread (1.8 and 1.3 vs flat's 2.1 at 20,000) — premature convergence, matching their plateaus; but aligned keeps the *most* spread (3.1, above flat) and still only ties random, its variety never concentrating on good cells. Flat holds an intermediate, productive spread and wins. (We had expected the cleaner "diversity ranks the operators"; the measurement falsifies it, and we report the falsification.) (ii) Varying *population* at fixed budget sweeps the diversity available to crossover, and the comparison moves with it (Table 3): at population 8 every arm — flat included — loses to random ($z < 0$); the gain rises monotonically until at 64 all four beat it. Crossover here is better than random exactly when it has enough diversity *and* an operator that turns diversity into good offspring rather than collapsing or dissipating it.

6 Relation to prior work

Nothing here is new in framing. Building-block recombination and crossover are Holland [2]; deceptive traps as their canonical test are Deb and Goldberg [3]; random search as a strong NAS baseline is [4, 9]; mutation-only evolutionary NAS is [7]; the competing-conventions problem the aligned operator targets motivates NEAT's historical markings [10] and He et al.'s modular inheritable crossover [11]; NAS-Bench-101 [12] makes a study at this seed count possible. The contribution is a clean, reproducible pairing: a positive control that shows exactly when crossover's building-block advantage appears, and a real NAS space where it does not — four structure-respecting crossovers all lose (consistently, though by small margins) to plain uniform crossover, two of them to random search — with a diversity mechanism that explains why.

7 Discussion

One rule ties the two testbeds together: crossover helps to the extent the space has separable, recombinable building blocks. The trap control makes the effect unmistakable — crossover's edge over mutation grows with the number of blocks until mutation cannot solve the problem. The real NAS cell does not supply such structure along its natural dimensions: recombining its wiring and labeling axes, aligning nodes by role, or transplanting whole paths all lose to a structure-blind uniform crossover, whose own edge over random is only diversity, which the structural priors collapse or dissipate. **On effect size:** none of the NAS-Bench-101 differences is large — every method lands within a few tenths of a percent of random, inside the benchmark's training noise, so no crossover meaningfully beats random search. Our claims are about the *consistent direction* of small effects resolved by thousands of paired seeds, not about magnitude; the practical message is the familiar one, that random search is very hard to beat on this space, sharpened by the fact that even the direction favors no cleverness. **Limitations:** one tabulated cell space with lookup fitness (no training confound, but no transfer guarantee to larger or hierarchical spaces); `aligned` is one alignment and a better one may exist — but the burden is now on exhibiting one that beats uniform crossover, which four natural attempts do not. The natural next step is a larger, hierarchical space where recombination has more room to matter.

Reproducibility

Every number regenerates from [13] with one command. The trap control is `bbtest.c` (`make bbtest`); the fitness table is extracted by `nb101_extract.c` (`make nb101_extract`); the search, oracle, and all operators are `validation/nb101.c` (`make nb101`), configurable by the environment (`POP`, `ELITE`, `SEEDS`, `BUDGETS`) and self-testing its oracle with `-selftest`. All build clean under `-std=c99 -pedantic` and run clean under AddressSanitizer and UBSan; a fixed xorshift generator makes every run reproducible.

References

[1] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks.* Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). `https://doi.org/10.5281/zenodo.20450526`

[2] J. H. Holland. *Adaptation in Natural and Artificial Systems.* University of Michigan Press, 1975.

[3] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.

[4] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.

[5] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.

[6] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

[7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.

[8] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.

[9] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.

[10] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.

[11] C. He, H. Tan, S. Huang, R. Cheng. Efficient evolutionary neural architecture search by modular inheritable crossover. *Swarm and Evolutionary Computation* 64, 2021.

[12] C. Ying, A. Klein, E. Real, E. Christiansen, K. Murphy, F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML* 2019.

[13] SMBPANN reference implementation. `https://github.com/Anode1/SMBPANN`.